REPUBLIQUE DU SENEGAL



Un Peuple – Un But – Une Foi

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE

UNIVERSITE GASTON BERGER DE SAINT-LOUIS



U.F.R DE SCIENCES APPLIQUEES ET DE TECHNOLOGIE

LICENCE INFORMATIQUE

Rapport de projet - développement d'Applications multi-tiers

SUJET : Conception et développement d'une application de gestion des projets tutorés

Frontend Angular · API Spring Boot · Persistance MySQL

Présenté par

Ahmadou Bamba DIAGNE P28 1039

Aïssatou GUYE P35 6741

Cheikh Loum P30 545

Année académique : 2025/2026

Introduction

Ce rapport décrit la conception, les fonctionnalités et la réalisation de l'application de gestion des projets tutorés. Il couvre les quatre profils utilisateurs, le parcours d'une candidature, les règles métier, l'architecture Angular–Spring Boot–MySQL, les principaux choix techniques et le lancement local. Le système permet de proposer des sujets, de candidater, de traiter les demandes et de consulter les affectations.

Le rapport met en regard les objectifs du sujet et les éléments présents dans le dépôt, puis décrit l'application dans son ensemble.

1. Contexte et objectifs

L'université souhaite centraliser la proposition des sujets de projets tutorés, la candidature des étudiants et les décisions d'affectation. L'application doit faciliter le suivi par les différents acteurs et démontrer une séparation claire entre présentation, logique métier et accès aux données.

Fonctionnalités attendues

Profil	**Besoin exprimé dans le sujet**
Administrateur	Gérer les étudiants et enseignants ; consulter les projets.
Enseignant	Créer, modifier et supprimer ses sujets ; consulter les candidatures reçues.
Étudiant	Consulter les sujets disponibles ; candidater ; suivre le statut.
Responsable pédagogique	Accepter ou rejeter les candidatures ; consulter les affectations.

Objectifs techniques

- Fournir des API REST JSON entre l'interface et le serveur.
- Centraliser les règles métier dans les services du backend.
- Contrôler l'accès aux ressources selon le rôle et le propriétaire.
- Conserver les utilisateurs, sujets et candidatures dans MySQL.

Périmètre réalisé

Le dépôt contient le frontend Angular, le backend Spring Boot, les services métier, les repositories JPA, les contrôleurs REST, la sécurité JWT, la configuration MySQL et un script de données de démonstration. Le frontend regroupe les écrans métier dans un portail et s'appuie sur des services HTTP pour communiquer avec l'API.

2. Architecture générale

Le navigateur sert les composants Angular. Ceux-ci appellent le backend au moyen de services HTTP, via des URL `/api`. En développement, le proxy Angular redirige ces appels vers le port 8089. Spring Security vérifie le jeton, le contrôleur délègue au service métier, puis le repository utilise JPA pour lire ou enregistrer les entités dans MySQL.

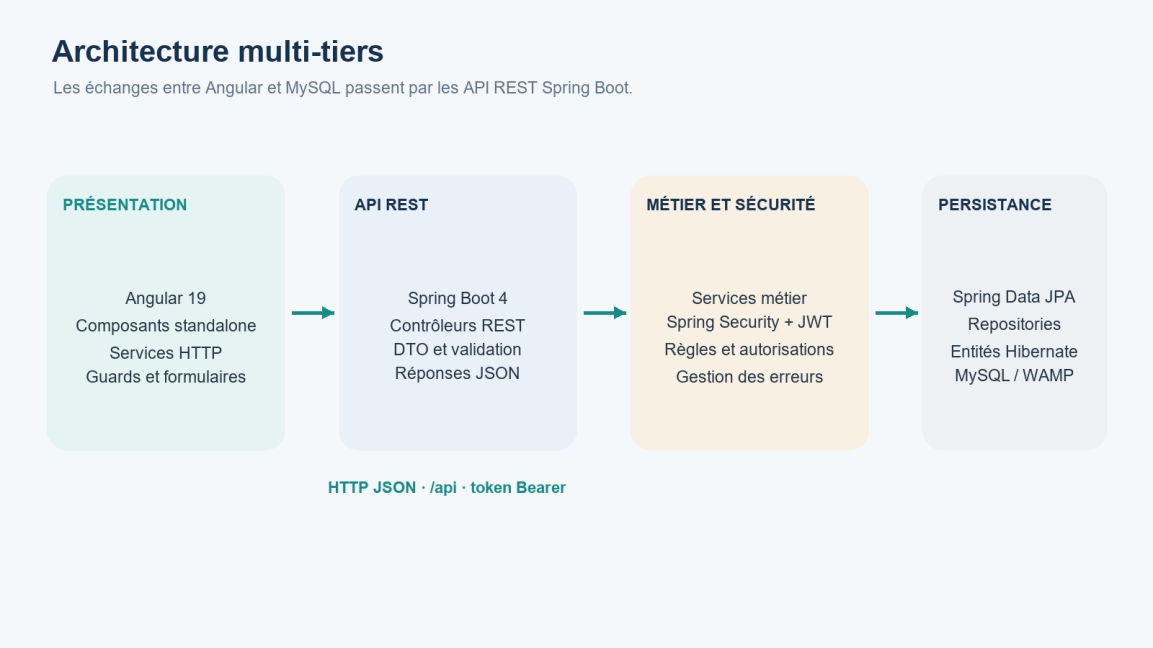


Figure 1 — Vue des couches et des échanges de l'application.

Rôle des couches

Couche	**Responsabilité dans le projet**
Présentation	Angular 19, vues, formulaires, navigation et affichage des réponses.
Communication	Services Angular dédiés aux ressources; requêtes HTTP JSON vers `/api`.
API	Contrôleurs Spring MVC, DTO, validation et codes HTTP.
Métier	Services Spring pour les sujets, candidatures, comptes et authentification.
Persistance	Repositories Spring Data JPA et entités Hibernate.
Données	MySQL; nom de base `gestionDeProjetBD` dans la configuration locale.

Cette séparation permet de modifier les règles métier sans les placer dans les composants de l'interface. Elle évite aussi une connexion directe du navigateur à la base de données.

3. Modèle de données

La classe abstraite `Utilisateur` porte les informations communes et les sous-classes représentent les quatre profils. La stratégie JPA JOINED conserve les informations communes dans la table `utilisateurs` et les attributs propres dans une table de profil. Un enseignant peut proposer plusieurs sujets; un sujet peut recevoir plusieurs candidatures; un étudiant peut déposer des candidatures sur des sujets différents.

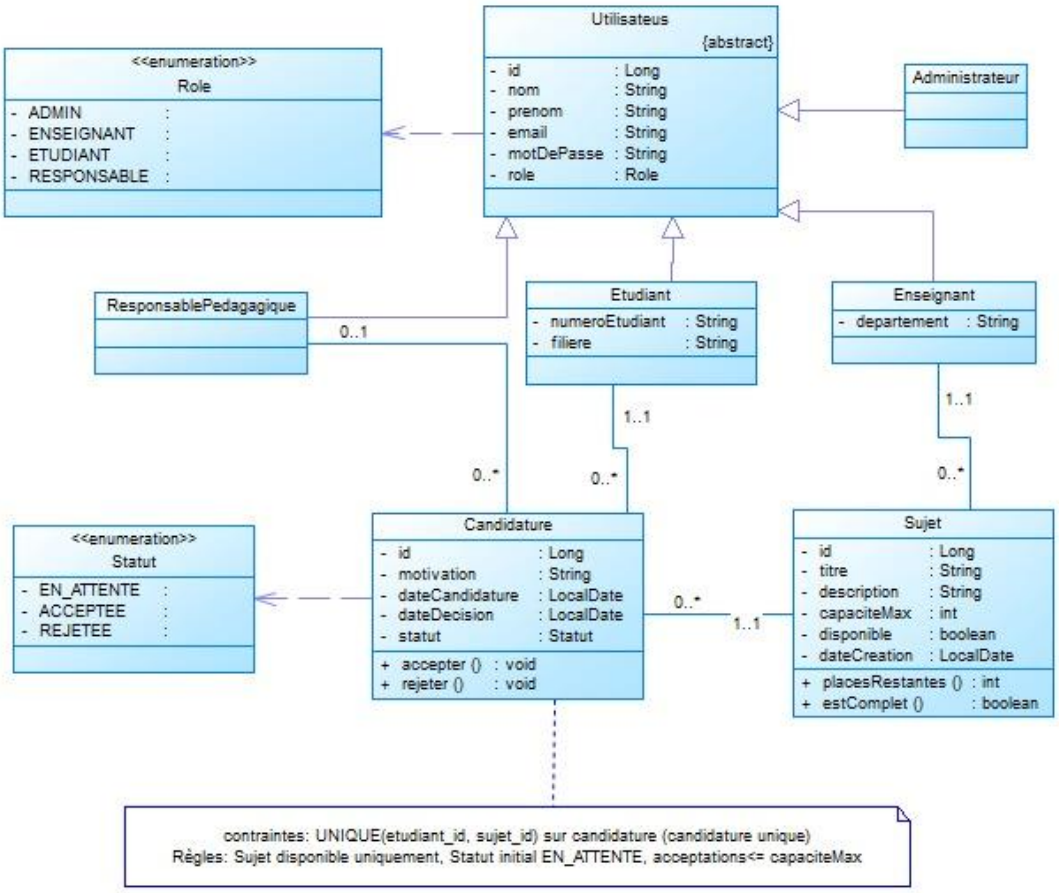


Figure 2 – Diagramme de classes de l'application.

Entités principales

Entité	**Données et relations**
Utilisateur	Identité, email unique, mot de passe chiffré et rôle.
Étudiant	Numéro étudiant unique et filière; hérite de Utilisateur.
Enseignant	Département; propriétaire de ses sujets.
Sujet	Titre, description, capacité, disponibilité et enseignant.
Candidature	Étudiant, sujet, date, motivation, statut, date et responsable de décision.
Responsable	Profil de décision associé à une candidature traitée.

Une contrainte d'unicité sur `(etudiant_id, sujet_id)` empêche les doublons au niveau de la base. Le backend effectue aussi une vérification métier avant l'enregistrement afin de renvoyer un message d'erreur compréhensible.

4. Backend et règles métier

Le backend est un projet Gradle Spring Boot configuré pour Java 21. Les contrôleurs définissent les ressources REST et délèguent les opérations aux services. Les DTO séparent les données échangées des entités JPA. Les validations et exceptions sont converties en réponses JSON communes.

API principales

Ressource	Opérations principales	Accès fonctionnel
Authentification	POST `/api/auth/login`	Connexion et remise du JWT.
Étudiants	GET, POST `/api/etudiants`; PUT/DELETE `/{id}`	Administration des comptes étudiants.
Enseignants	GET, POST `/api/enseignants`; PUT/DELETE `/{id}`	Administration des comptes enseignants.
Sujets	GET `/api/sujets`, `/disponibles`, `/mes-sujets`; POST/PUT/DELETE	Lecture selon profil; gestion par enseignant propriétaire.
Candidatures	POST; GET `/mes-candidatures`, `/sujet/{id}`, `/api/candidatures`	Dépôt étudiant; lecture par enseignant ou responsable.
Décisions	PUT `/{id}/accepter`, `/{id}/rejeter`	Décision par responsable pédagogique.
Affectations	GET `/api/affectations`	Consultation des candidatures acceptées.

Règles métier appliquées

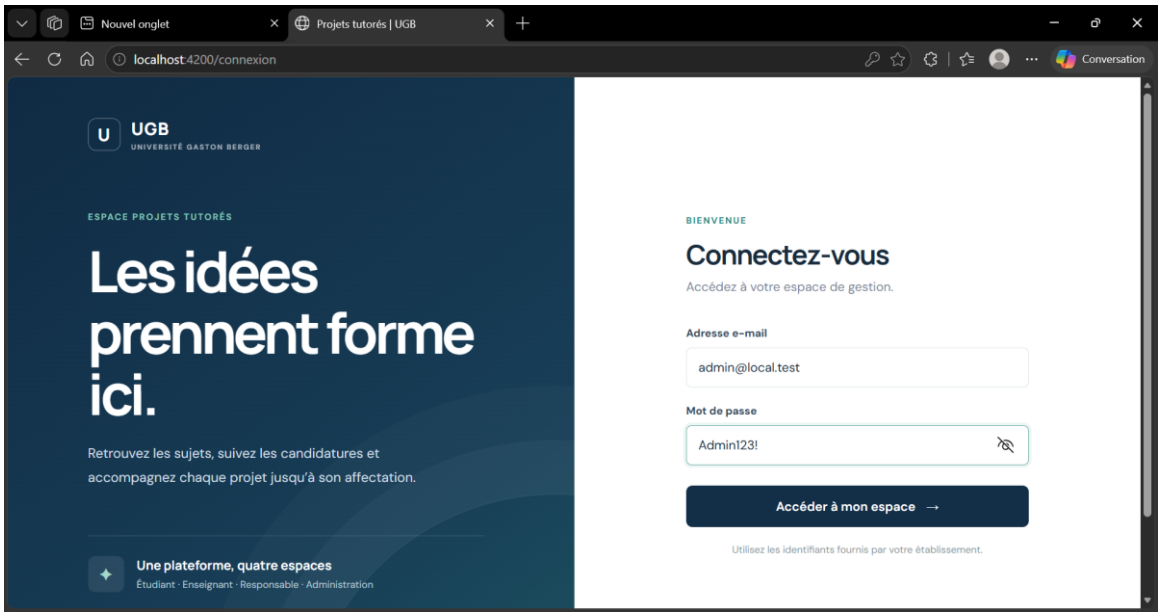
- Une seule candidature par étudiant et par sujet; une candidature nouvelle commence à EN_ATTENTE.
- Seuls les sujets disponibles acceptent une candidature.
- Seul le propriétaire enseignant peut modifier ou supprimer un sujet; la suppression est refusée si des candidatures y sont rattachées.
- Le responsable traite uniquement une candidature EN_ATTENTE; une candidature déjà traitée déclenche un conflit.
- L'acceptation est bloquée lorsque la capacité maximale est atteinte. Si la capacité est atteinte après acceptation, le sujet devient indisponible.
- Les affectations correspondent aux candidatures au statut ACCEPTEE.

Erreurs et validation

Les requêtes sont validées à l'entrée avec Jakarta Validation. Le gestionnaire global traduit les erreurs en réponses `ApiError` cohérentes : 400 pour une demande invalide, 401 pour des identifiants ou un jeton invalides, 403 pour un accès interdit, 404 pour une ressource absente et 409 pour un doublon ou un conflit métier.

5. Sécurité et intégration

L'authentification vérifie l'adresse électronique et le mot de passe avec Spring Security. Les mots de passe sont encodés avec BCrypt. Après connexion, le backend délivre un JWT signé, avec une durée d'expiration configurable.



Capture 1 – Connexion.

Mécanisme	**Mise en œuvre**
Authentification	POST '/api/auth/login'; retour du jeton, de l'identifiant et du rôle.
Autorisation	Spring Security, règles d'URL, `@PreAuthorize` et contrôles de propriété dans les services.
Navigation	Guard Angular de session et vérification du rôle lorsqu'une route porte une liste de rôles.
CORS	Origines locales déclarées dans 'application.properties'.
Proxy de développement	'frontend/src/proxy.conf.json' transmet '/api' à `http://localhost:8089`.
Documentation API	SpringDoc expose l'interface Swagger et les schémas OpenAPI.

Données de démarrage

Au lancement, les initialiseurs créent un administrateur et un responsable s'ils sont absents. Le README regroupe les identifiants locaux par défaut et explique comment les remplacer. Le script 'database/seed_demo.sql' ajoute des comptes de démonstration, des sujets et des candidatures; il suppose que le backend a déjà créé la base et les tables.

Pour un déploiement, les mots de passe et la clé JWT de développement doivent être remplacés par des valeurs gérées hors du dépôt. Les paramètres actuels décrivent une configuration locale WAMP/MySQL, pas un profil de production.

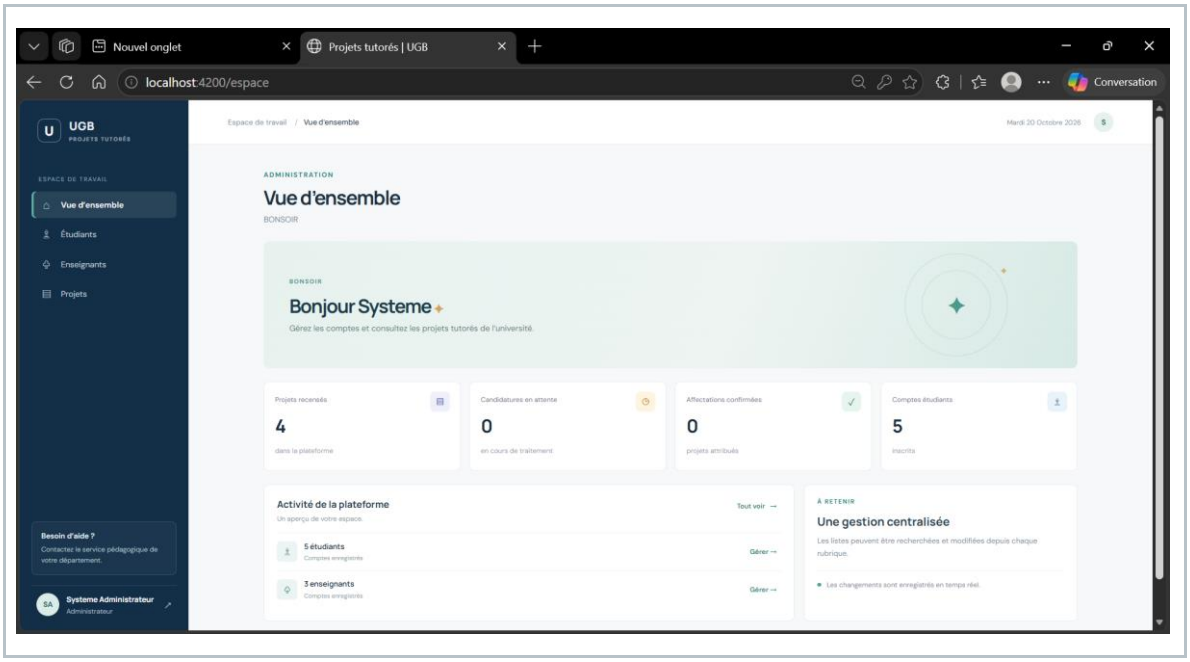
6. Frontend Angular

L'application Angular est construite avec des composants standalone. `LoginComponent` gère la connexion; `PortalComponent` fournit le portail et adapte le menu, les statistiques et les listes au rôle courant. Les services HTTP centralisent les échanges avec les ressources backend.

Organisation du code

Répertoire	**Rôle**
`frontend/src/app/auth/`	Formulaire de connexion et affichage des erreurs.
`frontend/src/app/portal/`	Vue d'ensemble, sujets, candidatures, comptes et affectations.
`frontend/src/app/core/`	Services API, authentification, modèles, guard et intercepteur.
`frontend/src/environments/`	URL de base de l'API utilisée par le frontend.

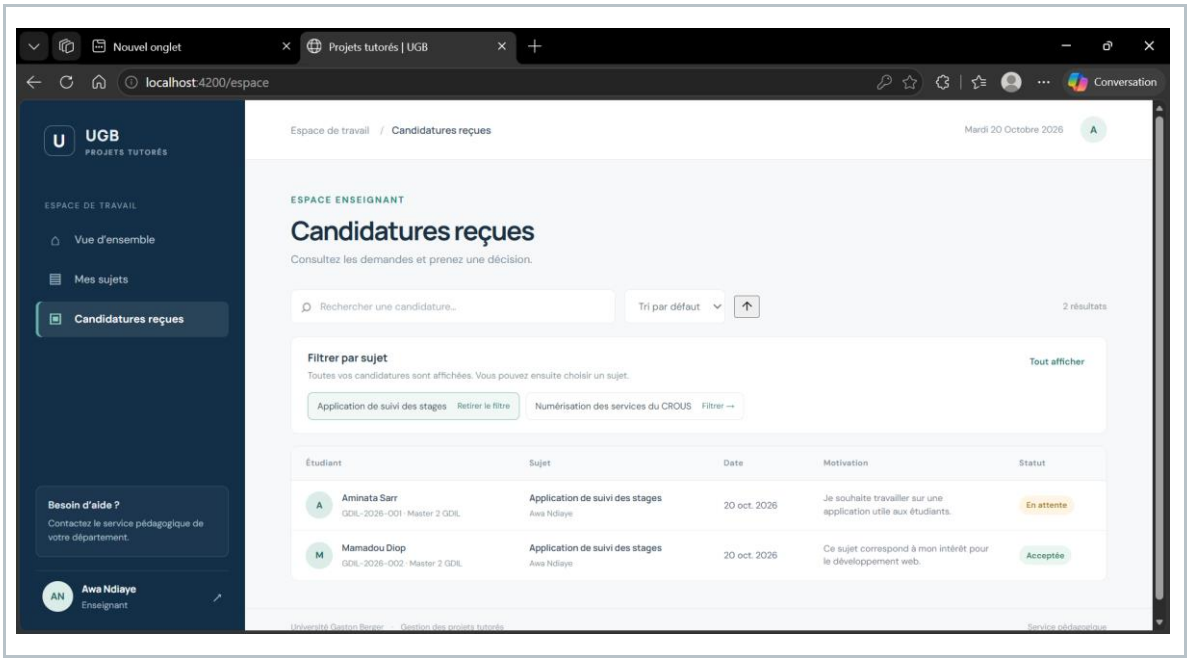
Fonctions d'interface



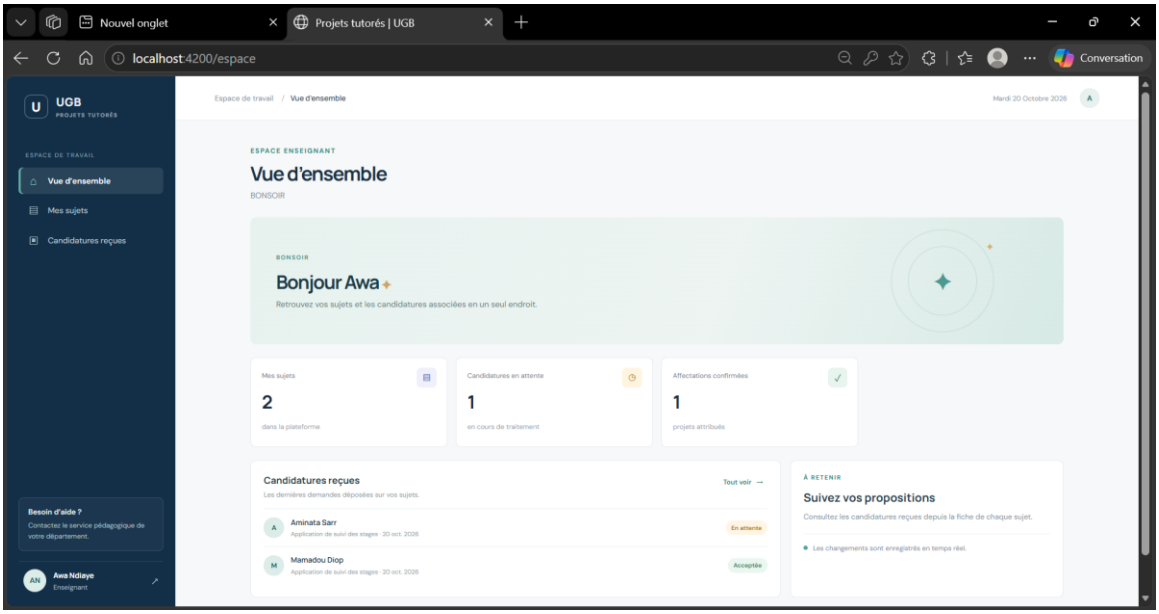
Capture 2 – Vue d'ensemble administrateur.

- Les formulaires de compte et de sujet appliquent des validations réactives; la candidature permet une motivation facultative.
- Les tableaux proposent recherche, tri et pagination. Le responsable peut filtrer les candidatures par statut.

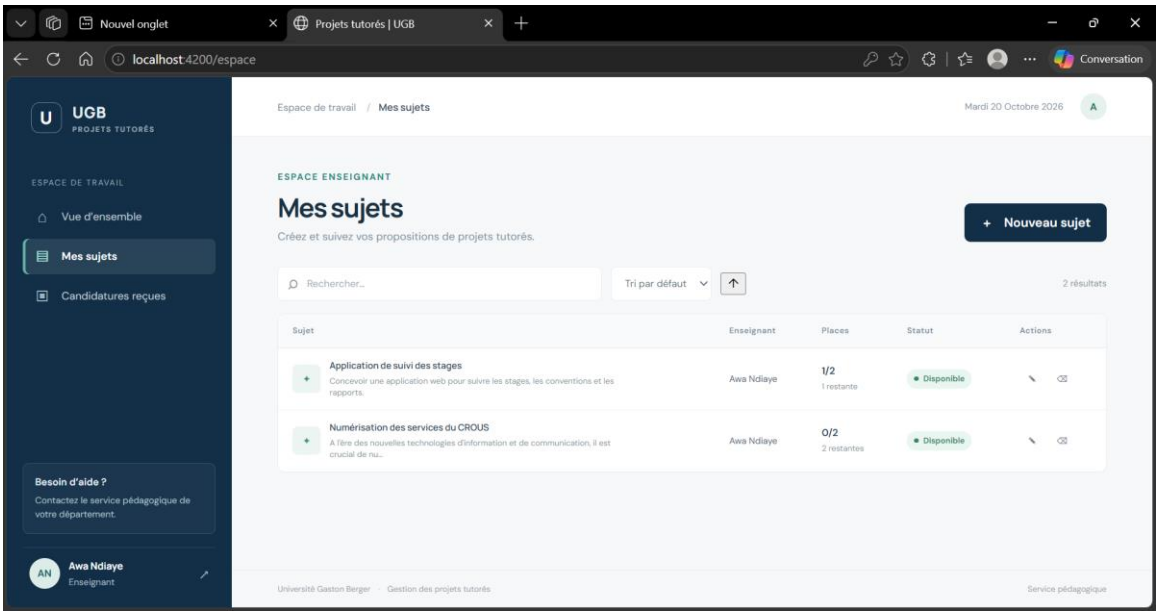
- L’enseignant voit les candidatures reçues pour tous ses sujets; il peut ensuite filtrer localement par sujet.



Capture 3 – Candidatures reçues par l’enseignant.

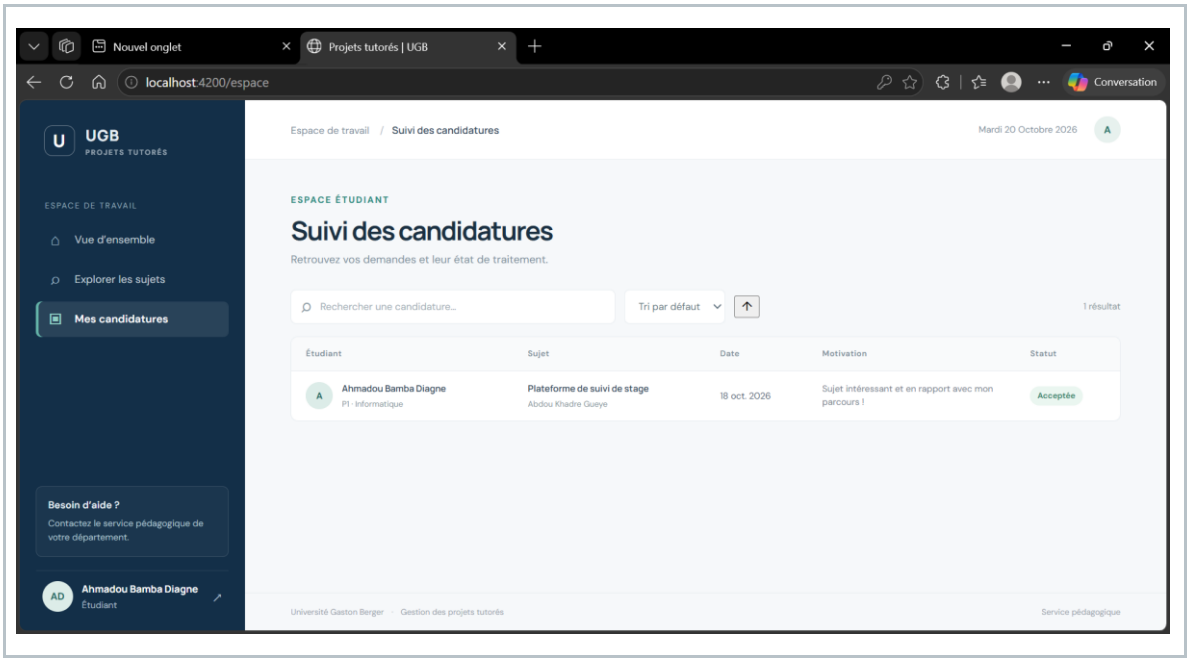


Capture 4 – Espace enseignant – vue d’ensemble.

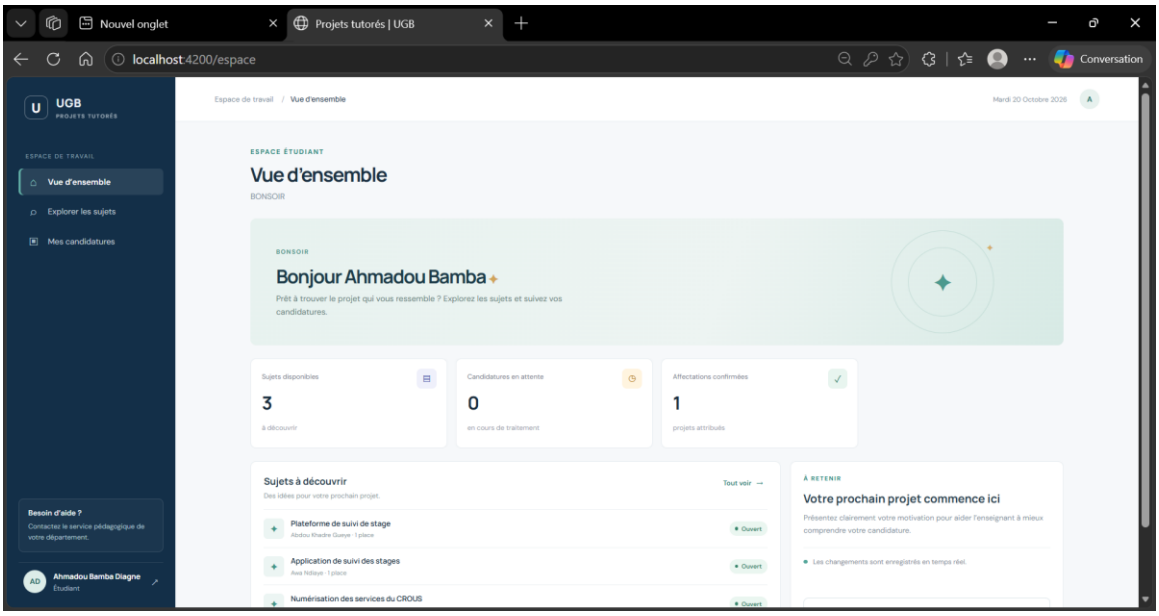


Capture 5 – Espace enseignant – sujets proposés.

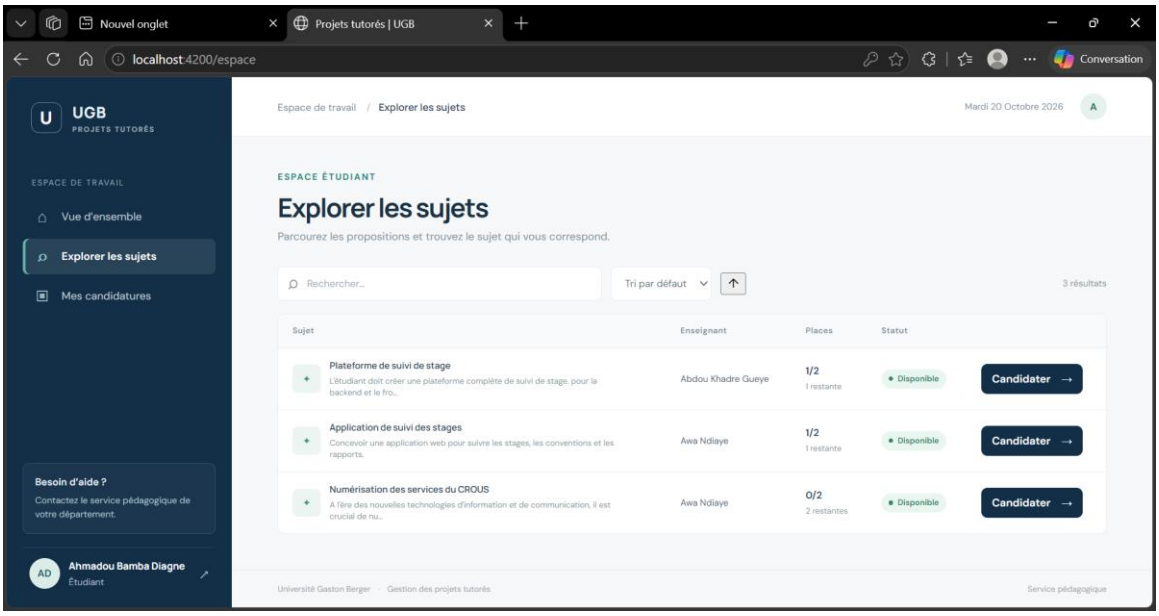
- L'étudiant consulte les sujets ouverts et l'état de ses demandes; le responsable consulte les candidatures et les affectations.



Capture 6 – Suivi de l'étudiant.

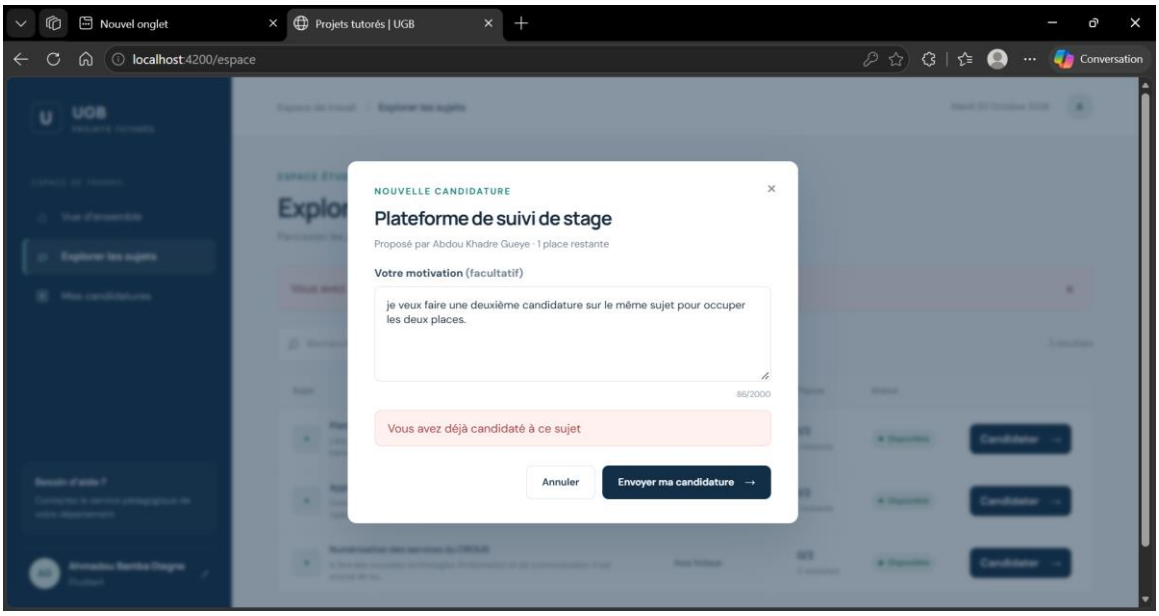


Capture 7 – Espace étudiant – vue d'ensemble.



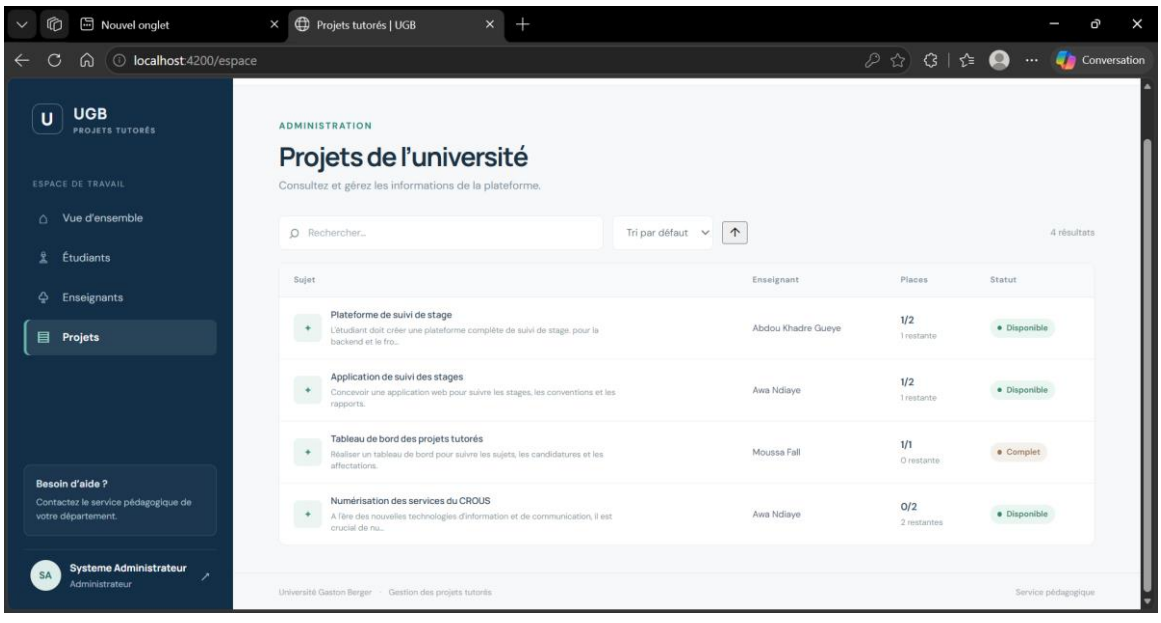
Capture 8 – Espace étudiant – sujets disponibles.

- L’étudiant tente de candidater sur un sujet auquel il a déjà candidaté

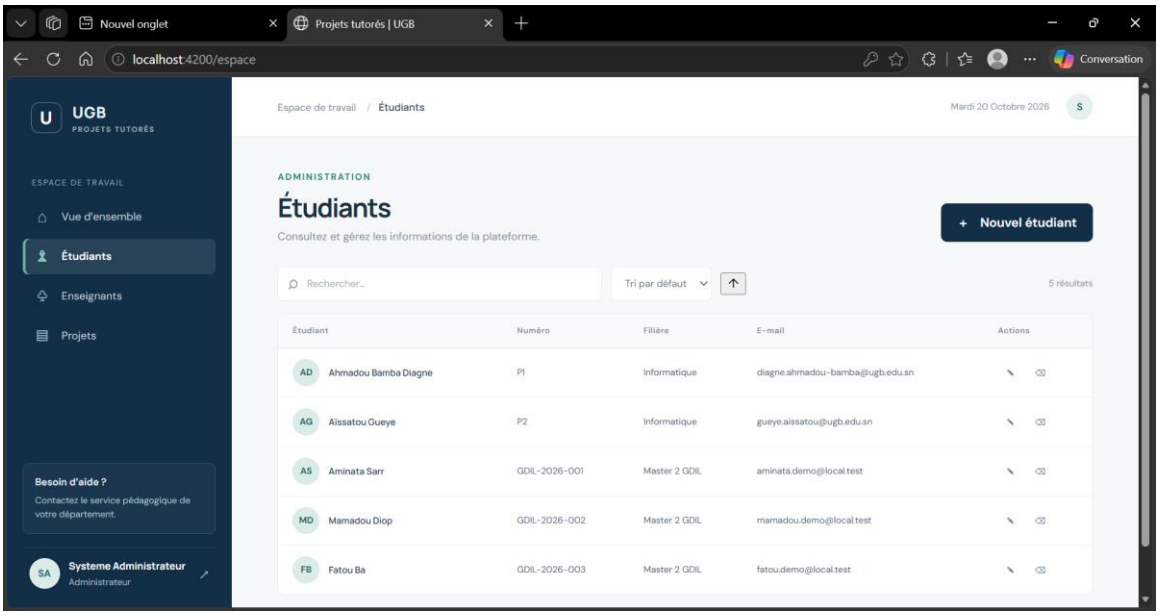


Capture 9 – Espace étudiant – candidature déjà déposée.

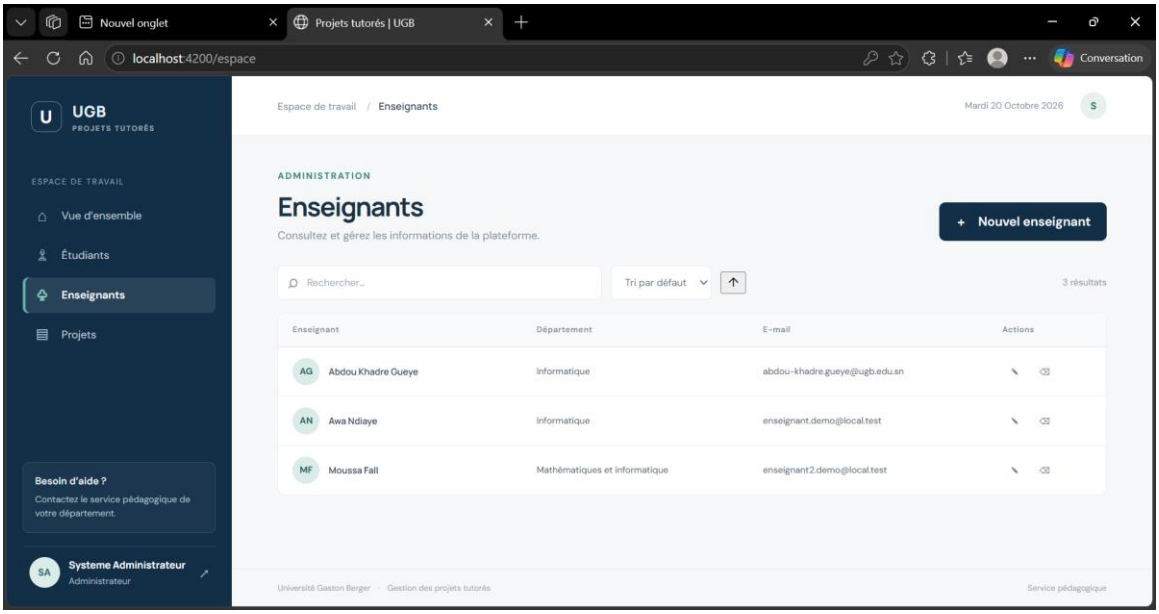
- L’administrateur gère les comptes étudiants et enseignants et consulte la liste des projets.



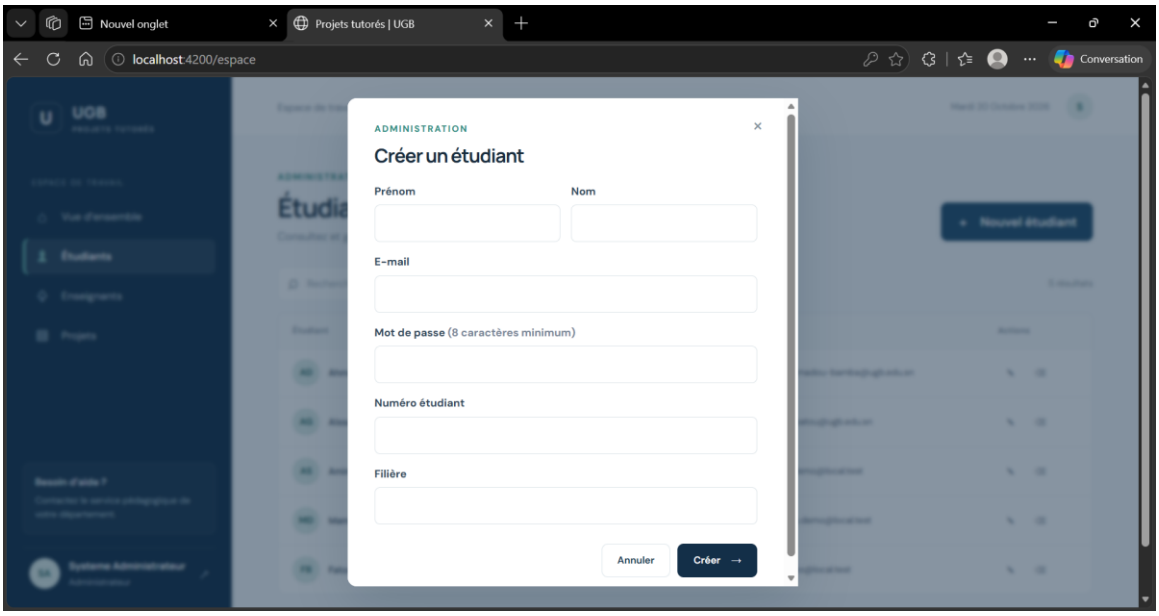
Capture 10 – Catalogue des sujets de projets.



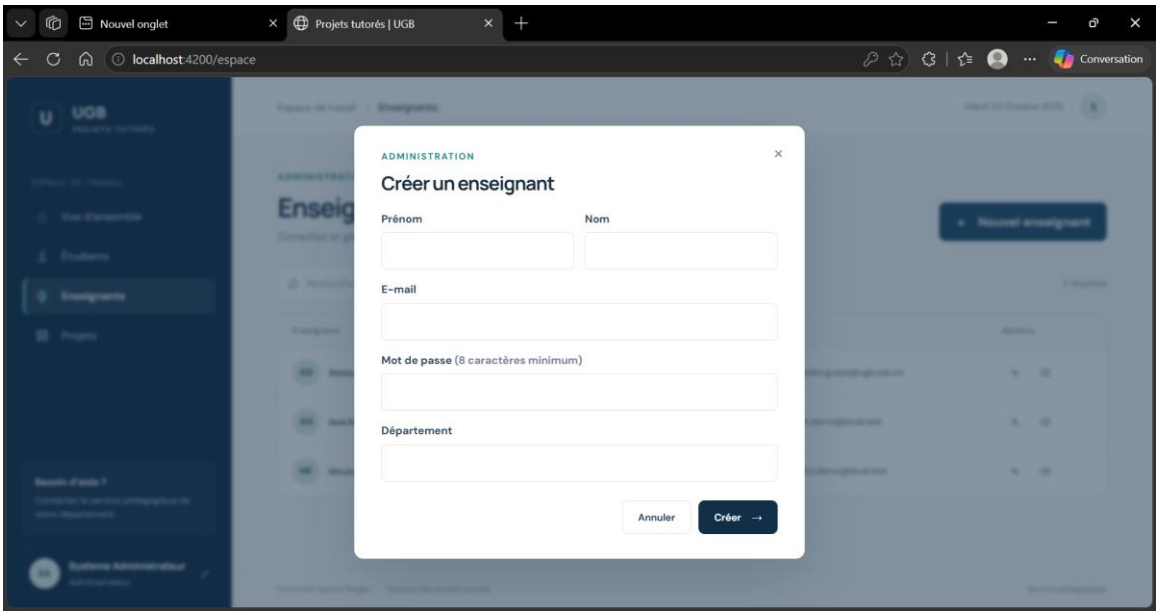
Capture 11 – Administration – liste des étudiants.



Capture 12 – Administration – liste des enseignants.



Capture 13 – Administration – création d'un compte étudiant.



Capture 14 – Administration – création d'un compte enseignant.

Le portail adapte ses vues au rôle connecté; les services HTTP, modèles et contrôles de navigation sont regroupés dans la couche Angular dédiée aux fonctions transverses.

7. Parcours de bout en bout

Le scénario met en relation les quatre profils et permet de comprendre le rôle de chaque couche. Le sujet de cadrage prévoit la création des comptes, la publication d'un sujet, la candidature, la décision et le suivi du résultat.

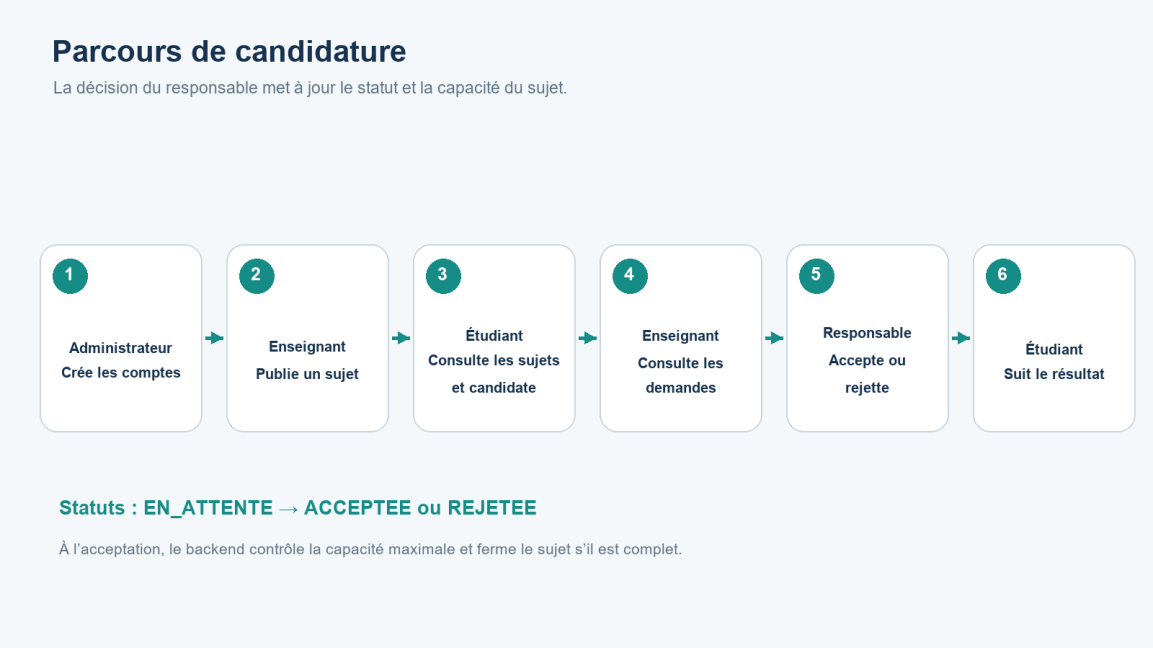
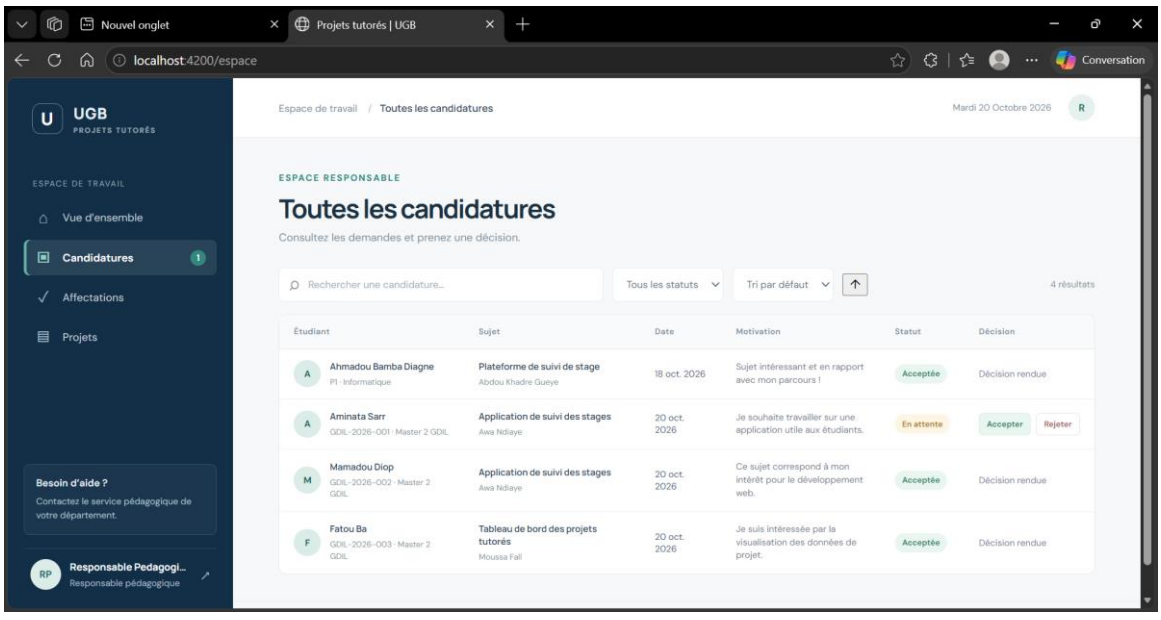
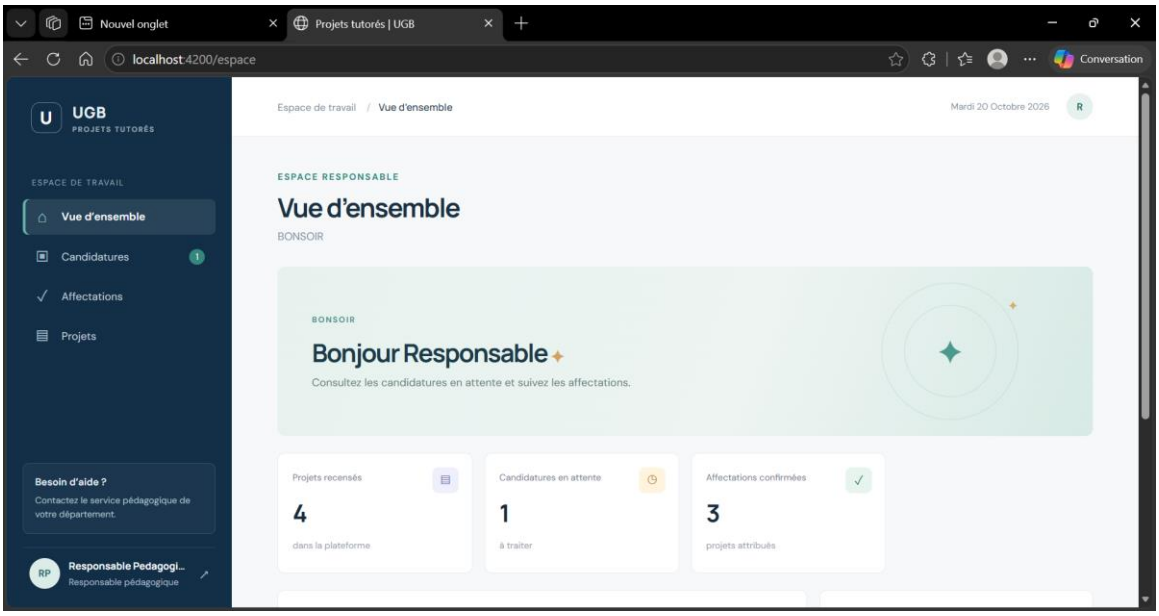


Figure 3 — Étapes métier du dépôt de candidature jusqu'au suivi du statut.

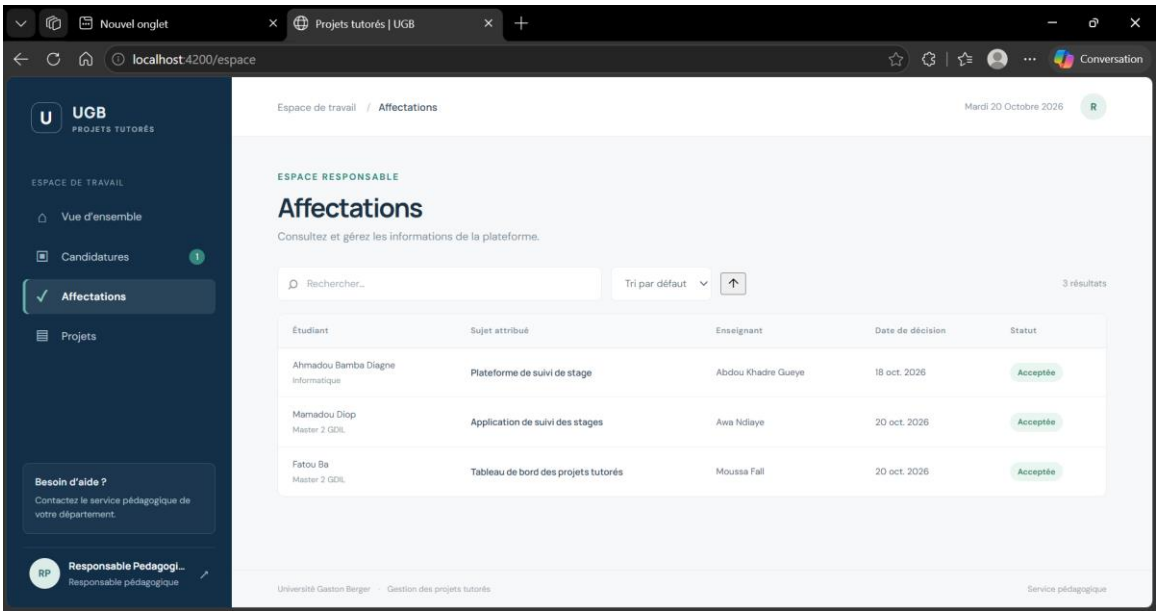
Échanges entre les acteurs



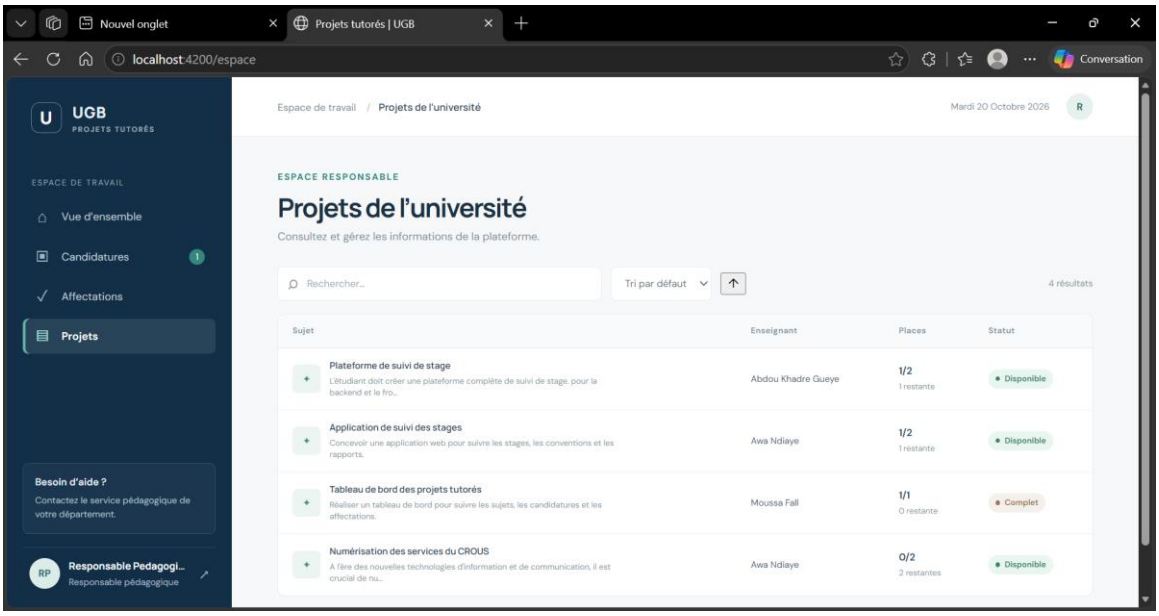
Capture 15 – Traitement des candidatures par le reponsable pédagogique.



Capture 16 – Espace responsable pédagogique – vue d'ensemble.



Capture 17 – Espace responsable pédagogique – affectations.



Capture 18 – Espace responsable pédagogique – projets.

Étape	Écran et action	Traitement backend
1	L'administrateur crée les comptes enseignant et étudiant.	Contrôle l'unicité et encode les mots de passe.
2	L'enseignant publie un sujet et sa capacité.	Associe le sujet à l'enseignant connecté.
3	L'étudiant consulte les sujets et envoie une candidature.	Vérifie disponibilité et absence de doublon; fixe EN_ATTENTE.
4	L'enseignant consulte les candidatures de ses sujets.	Le service vérifie la propriété de chaque sujet.
5	Le responsable accepte ou rejette la demande.	Vérifie le statut et la capacité; conserve la décision.
6	L'étudiant consulte le nouveau statut.	Le frontend recharge ses candidatures depuis l'API.

La couche Angular ne décide pas si une candidature est conforme. Elle présente les contrôles et messages; le service métier applique les règles faisant autorité, même si une requête est envoyée hors interface.

8. Installation et vérification

Lancement local

Étape	Action
Base	On démarre MySQL dans WAMP. Les paramètres locaux utilisent `root` et la base `gestionDeProjetBD`.

Backend	Depuis `backend/`, on lance `./gradlew bootRun` sous Windows avec `gradlew.bat`. Port : 8089. Java 21 est requis par la toolchain Gradle.
Frontend	Depuis `frontend/`, nous lançons `npm install`, puis `npm start`. L'interface est disponible sur `http://localhost:4200`.
Démo	Après le premier démarrage backend, il faut importer `database/seed_demo.sql` dans phpMyAdmin pour avoir des exemples.

9. Bilan

Le dépôt met en place les principaux éléments d'une application multi-tiers : interface Angular, API REST Spring, règles métier en services, repositories JPA et persistance MySQL. Les profils disposent de parcours dédiés et l'enseignant peut consulter les candidatures reçues. Les opérations sensibles et les contraintes de capacité sont contrôlées côté serveur.

Conclusion

L'application couvre le cycle d'un projet tutoré depuis la publication d'un sujet jusqu'à la décision de candidature et à la consultation des affectations. L'architecture sépare présentation, API, logique métier et persistance; les règles importantes sont contrôlées côté serveur.